Table of Contents

11. References & Technical Stack

1. Executive Summary

This report documents the complete technical work of **Phase 3** of the Medical Image Diagnosis System project, developed at Cairo University, Faculty of Computers and Artificial Intelligence, as part of the Supervised Learning course (Spring 2026). Phase 3 extends the project's deep learning pipeline by introducing **Recurrent Neural Networks (RNN, GRU, LSTM)** and a **Vision Transformer (ViT)** architecture for binary classification of chest radiographs sourced from the NIH Chest X-Ray dataset.

Two Jupyter notebooks comprise the Phase 3 deliverables:

- **Notebook 1 — EfficientNetB0 + BiRNN / BiGRU (Fine-Tuned):** A hybrid CNN–sequence architecture where the EfficientNetB0 backbone extracts spatial feature maps, which are reshaped into patch sequences and processed by Bidirectional RNN or GRU layers. A two-stage fine-tuning strategy is applied.
- **Notebook 2 — BiLSTM & Vision Transformer:** Two independent approaches: (a) a pure Bidirectional LSTM treating each X-ray as a sequence of pixel rows, and (b) a fine-tuned ViT-Base/16 model with attention heatmap visualization implemented in PyTorch using the *timm* library.

Phase 2 established the following baselines on the same NIH dataset:

Model	Accuracy	F1-Score	AUC-ROC
Phase 2: CNN (from scratch)	0.614	0.621	0.648
Phase 2: VGG16 Transfer Learning (Best)	0.667	0.656	0.705
Phase 2: Autoencoder (Unsupervised)	0.550	0.130	0.480

Table 1: Phase 2 Baseline Results

Phase 3 builds directly on these results, aiming to determine whether sequence and attention-based architectures provide measurable improvements in radiological diagnostic accuracy over convolutional baselines.

2. Project Overview & Context

2.1 Project Goal

The Medical Image Diagnosis System is a multi-phase academic research project investigating the application of modern deep learning architectures to automated chest X-ray pathology detection. The system performs **binary classification**: distinguishing radiographs with no finding (Normal) from those exhibiting one or more pathological findings (Disease). This task mirrors a real-world clinical triage scenario where radiologists must rapidly screen large volumes of chest X-rays.

2.2 Phase Progression

Phase	Focus	Key Techniques
Phase 1	Data Exploration & Preprocessing	EDA, normalization, augmentation pipeline
Phase 2	CNN & Transfer Learning	Custom CNN, VGG16, Autoencoder
Phase 3 (this report)	Sequence & Transformer Models	BiRNN, BiGRU, BiLSTM, ViT-Base/16

Table 2: Project Phase Progression

2.3 Phase 3 Objectives

Objective 1: Implement and train Bidirectional RNN (SimpleRNN) and GRU models on top of a pretrained EfficientNetB0 backbone with two-stage fine-tuning on NIH Chest X-Ray images.

Objective 2: Implement and train a pure Bidirectional LSTM model treating each X-ray image as a temporal sequence of pixel rows (128 time steps × 128 features).

Objective 3: Fine-tune the Vision Transformer (ViT-Base/16) pretrained on ImageNet for binary chest X-ray classification using PyTorch and the timm library.

Objective 4: Generate and analyze attention heatmaps from the ViT's last transformer block, visualizing which anatomical regions the model attends to.

Objective 5: Perform a comprehensive cross-phase comparison of all models (Phase 2 & 3) using Accuracy, F1-Score, and AUC-ROC metrics.

3. Dataset Description

3.1 NIH Chest X-Ray Sample Dataset

The dataset used throughout Phase 3 is the **NIH Chest X-Ray Sample**, a publicly available subset of the full NIH Clinical Center Chest X-Ray dataset released by the National Institutes of Health. The sample subset comprises **5,606 chest radiograph images** in PNG format (1024 × 1024 resolution), distributed across 14 possible pathological finding labels and one "No Finding" category. For the binary classification task, all non-"No Finding" images are grouped into a single "Disease" class.

Property	Value
Source	NIH Clinical Center (Kaggle: nih-chest-xrays/sample)
Total Images	5,606
Image Format	PNG (grayscale, 1024 × 1024)
Labels	14 pathologies + "No Finding"
Task	Binary Classification (Normal vs Disease)
Normal ("No Finding")	~60% of dataset
Disease (any pathology)	~40% of dataset
Modality	Chest X-Ray (frontal view)

Table 3: NIH Chest X-Ray Sample Dataset Properties

3.2 Data Preprocessing Pipeline

Both notebooks implement comparable preprocessing pipelines tailored to their respective model requirements. The key steps are:

- **Image Loading:** Images are loaded from disk using the PIL library (Python Imaging Library) and converted to the appropriate color mode — grayscale ("L") for the LSTM model and RGB ("RGB") for the ViT and EfficientNet-based models.
- **Resizing:** Notebook 1 resizes all images to **224 × 224** pixels (required by EfficientNetB0's pretrained weights). Notebook 2 uses **128 × 128** for the LSTM and **224 × 224** for the ViT.
- **Normalization:** Notebook 1 applies EfficientNet-specific preprocessing via preprocess_input(). Notebook 2 normalizes pixel values to [0,1] for LSTM, and applies ImageNet mean/std normalization ([0.485,0.456,0.406] / [0.229,0.224,0.225]) for the ViT.
- **Label Encoding:** "No Finding" images receive label 0 (Normal); all other findings receive label 1 (Disease), regardless of the specific pathology type.
- **Class Imbalance Handling:** Both notebooks compute balanced class weights using sklearn's compute_class_weight() to penalize the majority class during training, compensating for the imbalanced class distribution.

3.3 Train / Validation / Test Split

Both notebooks apply an identical stratified splitting strategy with random seed=42 to ensure reproducibility and fair cross-notebook comparison:

Split	Proportion	NB1 (5,606 imgs)	NB2 (3,000 imgs)
Training	70%	≈ 3,924	≈ 2,100
Validation	15%	≈ 841	≈ 450
Test	15%	≈ 841	≈ 450

Table 4: Dataset Split Configuration

Stratified splitting is applied at each step to maintain the original class distribution across all three subsets, preventing data leakage and ensuring that evaluation metrics reflect true generalization performance.

4. Notebook 1: EfficientNetB0 + BiRNN / BiGRU

File: phase3_nih_xray_rnn_gru_finetuned_copy.ipynb

This notebook implements a hybrid CNN-sequence architecture combining the EfficientNetB0 convolutional backbone with Bidirectional Recurrent layers (both SimpleRNN and GRU variants). A two-stage fine-tuning approach is employed to maximize transfer of pretrained ImageNet features to the medical imaging domain.

4.1 Architecture Overview

The model design follows a principled motivation for applying sequence models to 2D images: the EfficientNetB0 backbone produces a spatial feature map of shape **(H, W, C)**, which is reshaped into a sequence of **H×W visual patches**, each with C feature channels. The recurrent layer then learns sequential relationships between these spatial patches, analogous to how transformers learn patch relationships via attention.

Layer / Block	Output Shape	Description
Input	(224, 224, 3)	Grayscale X-ray converted to RGB
EfficientNetB0 (frozen/unfrozen)	(7, 7, 1280)	Pretrained ImageNet feature extractor; no top
Conv2D(256) + BN	(7, 7, 256)	Channel reduction with 1×1 conv + batch norm
Reshape (H×W, C)	(49, 256)	Feature map converted to 49-step sequence
Bidirectional RNN/GRU(128)	(49, 256)	return_sequences=True, dropout=0.25
GlobalAveragePooling1D	(256,)	Aggregate sequence into fixed vector
Dense(128) + BN + Dropout(0.45)	(128,)	ReLU activation, regularization
Dense(64) + Dropout(0.30)	(64,)	Secondary dense layer
Dense(1, sigmoid)	(1,)	Binary probability output

Table 5: EfficientNetB0 + BiRNN/BiGRU Architecture

4.2 Two-Stage Fine-Tuning Strategy

Fine-tuning pretrained models for medical imaging requires care to avoid catastrophic forgetting while allowing domain-specific adaptation. The notebook implements a two-stage protocol:

- **Stage 1 — Head Training (8 epochs):** The EfficientNetB0 backbone is fully frozen (all layers non-trainable). Only the channel reduction Conv2D, Bidirectional RNN/GRU, and dense classification head are trained. Learning rate = 1e-4 (Adam optimizer). This lets the recurrent head converge without disrupting pretrained feature representations.
- **Stage 2 — Partial Backbone Fine-Tuning (6 epochs):** The last **25 layers** of the EfficientNetB0 backbone are unfrozen, while earlier layers remain frozen. BatchNormalization layers remain frozen throughout fine-tuning (standard practice for small medical datasets). Learning rate is reduced to **1e-5** to prevent large weight updates that could damage pretrained representations.

4.3 TF Data Pipeline & Data Augmentation

A high-performance TensorFlow tf.data pipeline is constructed for efficient training. The following augmentations are applied **only to the training set** to improve generalization without introducing unrealistic transformations for chest X-rays:

Augmentation	Parameter	Justification
RandomFlip (horizontal)	—	Left-right flip is anatomically plausible for chest X-rays
RandomRotation	±4%	Small rotations simulate patient positioning variation
RandomZoom	8%	Slight scale changes handle varying patient-to-detector distance
RandomContrast	10%	Contrast variation simulates different X-ray exposure settings

Table 6: Training Data Augmentation Strategy

4.4 Training Configuration & Callbacks

Hyperparameter	Stage 1	Stage 2
Optimizer	Adam	Adam
Learning Rate	1e-4	1e-5
Loss Function	Binary Crossentropy	Binary Crossentropy
Batch Size	32	32
Epochs	8 (max)	6 (max)
Class Weights	Balanced	Balanced
Image Size	224 × 224	224 × 224

Table 7: Notebook 1 Training Hyperparameters

Three training callbacks govern the learning dynamics:

- **ModelCheckpoint:** Saves the best model weights based on validation AUC (monitor='val_auc', mode='max', save_best_only=True). Separate checkpoints are maintained for Stage 1 and Stage 2.
- **EarlyStopping:** Stops training if validation AUC does not improve for **4 consecutive epochs**, restoring the best weights found.
- **ReduceLROnPlateau:** Reduces learning rate by factor 0.3 when validation loss plateaus for 2 epochs, with minimum LR=1e-7.

4.5 BiRNN Model — Architecture Detail & Training

The Bidirectional SimpleRNN variant uses two-directional processing of the 49-patch visual sequence. The recurrent units process the sequence forward (patch 1 → 49) and backward (patch 49 → 1) simultaneously, with both directions' outputs concatenated. The model name is EfficientNetB0_BiRNN.

Key architectural note: return_sequences=True is used in the recurrent layer, retaining the full sequence of hidden states. GlobalAveragePooling1D then averages across all 49 positions to produce a single 256-dimensional feature vector.

Monitoring Metrics:

Training monitors four metrics simultaneously: **Accuracy**, **Precision** (proportion of disease predictions that are correct), **Recall** (proportion of actual disease cases detected), and **AUC** (Area Under the ROC Curve — primary optimization target).

4.6 BiGRU Model — Architecture Detail & Training

The Bidirectional GRU variant replaces the SimpleRNN cell with a Gated Recurrent Unit cell. GRU addresses the vanishing gradient problem of vanilla RNNs through two gates: the **update gate** (controls how much previous state to retain) and the **reset gate** (controls how much previous state to forget when computing the candidate hidden state). This makes GRU more effective at capturing longer-range dependencies in the patch sequence.

The architecture is otherwise identical to the BiRNN variant, sharing the same backbone, reshape operation, dense head, dropout configuration, and training procedure. The model name is EfficientNetB0_BiGRU.

4.7 Evaluation Framework

Both models are evaluated on the held-out test set (never seen during training or validation). The evaluation function computes:

- **Accuracy:** Overall proportion of correct predictions (threshold = 0.5).
- **F1-Score:** Harmonic mean of Precision and Recall — particularly important for the imbalanced disease detection task.
- **AUC-ROC:** Area Under the Receiver Operating Characteristic curve, threshold-independent measure of discriminative ability.
- **Classification Report:** Per-class precision, recall, and F1 for both "No Finding" and "Disease" categories.
- **Confusion Matrix:** Visualized as a heatmap showing True Positives, True Negatives, False Positives, and False Negatives.
- **ROC Curve:** FPR vs TPR at all classification thresholds.

A final side-by-side bar chart compares all three metrics (Accuracy, F1, AUC) between the BiRNN and BiGRU models, and the comparison results are saved to phase3_nih_xray_rnn_gru_results.csv.

5. Notebook 2: BiLSTM & Vision Transformer (ViT)

File: phase3_lstm_vit.ipynb | Institution: Cairo University, Faculty of Computers and Artificial Intelligence

This notebook implements two architecturally distinct approaches to chest X-ray classification: a **Bidirectional LSTM** model using TensorFlow/Keras (treating X-rays as pixel-row sequences) and a **Vision Transformer (ViT-Base/16)** fine-tuned using PyTorch and the timm library. It also produces attention visualization heatmaps and a comprehensive cross-phase comparison.

5.1 BiLSTM Model — Architecture

The core insight of the LSTM approach is to treat each chest X-ray as a **temporal sequence of anatomical rows**. The image (128 × 128 grayscale) is reshaped from (N, 128, 128, 1) to (N, 128, 128) — each of the 128 rows becomes one time step, with 128 pixel values as the feature vector at that step. This framing has clinical intuition: the vertical axis of a chest X-ray transitions from the shoulder region (top) through the lung fields to the diaphragm (bottom), and the LSTM can potentially learn these anatomical transitions.

Layer	Configuration	Output Shape
Input	timesteps=128, features=128	(128, 128)
Bidirectional LSTM 1 (BiLSTM_1)	128 units, return_seq=True, dropout=0.2, recurrent_dropout=0.2	(128, 256)
BatchNormalization	—	(128, 256)
Bidirectional LSTM 2 (BiLSTM_2)	64 units, return_seq=True, dropout=0.2, recurrent_dropout=0.2	(128, 128)
BatchNormalization	—	(128, 128)
LSTM Final	32 units, return_seq=False, dropout=0.2	(32,)
BatchNormalization	—	(32,)
Dense(128, ReLU) + Dropout(0.5)	FC1 — heavy regularization	(128,)
Dense(64, ReLU) + Dropout(0.3)	FC2	(64,)
Dense(1, Sigmoid)	Binary output	(1,)

Table 8: BiLSTM Model Architecture (ChestXray_BiLSTM)

5.2 BiLSTM Training Configuration & Results

The LSTM model is compiled with Adam (lr=1e-3) and trained for up to **20 epochs** with the following callbacks: EarlyStopping (monitor=val_auc, patience=5), ReduceLROnPlateau (factor=0.5, patience=3), and ModelCheckpoint (save best by val_auc). Class weights are applied to account for the dataset's class imbalance.

Training Curves Visualized: Loss, Accuracy, AUC-ROC, and Precision/Recall are plotted across epochs in a 2×2 grid, with a vertical dashed line marking the best epoch. The figure is saved as lstm_training_curves.png.

The LSTM model directly competes with the Phase 2 CNN baseline (Acc=0.614, F1=0.621, AUC=0.648), with results printed alongside baseline values for immediate comparison.

5.3 Vision Transformer (ViT-Base/16) — Architecture

The Vision Transformer is implemented using **PyTorch** and the **timm** library, providing access to the pretrained vit_base_patch16_224 model (ViT-B/16). Unlike CNN-based models, ViT divides the input image into a fixed grid of non-overlapping patches and processes them as a sequence using transformer self-attention mechanisms.

Property	Value
Base Model	vit_base_patch16_224 (timm)
Pretraining	ImageNet-21k / ImageNet-1k
Input Size	224 × 224 × 3 (RGB)
Patch Size	16 × 16 pixels
Number of Patches	196 (14 × 14 grid)
Sequence Length	197 (196 patches + 1 CLS token)
Embedding Dimension	768
Transformer Blocks	12 total (blocks 0–11)
Attention Heads	12 heads per block
Total Parameters	~86.6 million

Table 9: ViT-Base/16 Specifications

Custom Classification Head:

The original ViT classification head is removed (num_classes=0) and replaced with a custom head designed for binary medical classification:

- LayerNorm(768) — stabilizes the CLS token features
- Linear(768 → 256) + GELU activation
- Dropout(0.4) — aggressive regularization for small dataset
- Linear(256 → 64) + GELU activation
- Dropout(0.2)
- Linear(64 → 1) — binary logit output (BCEWithLogitsLoss)

5.4 ViT Fine-Tuning Strategy

Selective layer unfreezing is applied to balance domain adaptation with preservation of pretrained representations:

- **Frozen:** Transformer blocks 0–7 (early layers encoding low-level visual features and general visual patterns from ImageNet)
- **Trainable:** Transformer blocks 8–11 (last 4 blocks, encoding high-level semantic features) + Layer Normalization
- **Custom head:** Fully trainable from random initialization
- **Trainable parameters:** Approximately 28 million (out of ~86.6M total, approximately 32%)

5.5 ViT Training Loop & Configuration

Unlike the TF/Keras-based models, the ViT uses a custom PyTorch training loop with explicit epoch iterations. Key training configuration:

Setting	Value	Rationale
Optimizer	AdamW	Weight decay (0.01) for ViT regularization
Learning Rate	1e-4	Conservative LR to preserve pretrained weights
LR Scheduler	CosineAnnealingLR	Smooth LR decay to eta_min=1e-6
Loss Function	BCEWithLogitsLoss	Numerically stable binary cross-entropy
Pos. Weight	class_weight[1]/class_weight[0]	Addresses class imbalance
Batch Size	16	Smaller batch for ViT due to memory constraints
Epochs	10	ViT converges faster than RNN models
Best Model	Saved by val_auc (best_vit.pth)	Checkpoint at peak AUC

Table 10: ViT Training Configuration

Data augmentation for ViT training uses: random horizontal flip, random rotation (±8°), ColorJitter (brightness ±0.2, contrast ±0.2), and ImageNet normalization. Validation uses only resize and normalization (no augmentation).

5.6 ViT Evaluation Results

After loading the best checkpoint (best_vit.pth), the ViT is evaluated on the test set. A confusion matrix and ROC curve are generated and saved as vit_evaluation.png. Results are compared directly against Phase 2 CNN and VGG16 baselines.

5.7 Attention Visualization — Heatmaps

One of the most significant contributions of the ViT notebook is its **attention heatmap visualization** — providing interpretable insight into which image regions drive the model's classification decisions.

Technical Implementation:

Attention maps are extracted from the **last transformer block's** attention dropout layer using PyTorch forward hooks. The hook captures the attention weight tensor of shape **(batch, 12 heads, 197, 197)** (197 = 196 patch tokens + 1 CLS token).

- **Step 1 — Extract CLS attention:** The CLS token's attention to all 196 patch tokens is isolated: attn[:, 0, 1:], yielding a (12, 196) tensor.
- **Step 2 — Average across heads:** The 12 attention heads are averaged to produce a single (196,) attention vector.
- **Step 3 — Reshape to grid:** The 196-vector is reshaped to a 14×14 spatial grid (matching the 14×14 patch layout for 224px / 16px patches).
- **Step 4 — Normalize & upsample:** The attention map is normalized to [0,1] and bicubically interpolated back to 224×224 for overlay.
- **Step 5 — Overlay:** The attention heatmap (jet colormap) is blended with the original X-ray (60% original + 40% heatmap) to create overlays.

Visualization Outputs:

Three visualization types are produced:

- **Normal vs Diseased Comparison:** 4 Normal + 4 Diseased test images shown in a 3-row × 8-column grid: original, attention map, and overlay. Predicted label and confidence are displayed per image. Saved as vit_attention_heatmaps.png.
- **Per-Head Visualization:** For a single diseased sample, all 12 attention heads from the last transformer block are visualized as individual overlays, revealing how different attention heads specialize in different anatomical regions. Saved as vit_attention_heads.png.

6. Cross-Phase Comparison: Phase 2 vs Phase 3

Notebook 2 produces a comprehensive comparison table encompassing all models from Phase 2 and Phase 3. The following table mirrors the structure of the notebook's comparison_df DataFrame:

Phase	Model	Type	Accuracy	F1-Score	AUC-ROC
Phase 2	CNN (from scratch)	CNN	0.614	0.621	0.648
Phase 2	VGG16 — Transfer Learning (Best)	Transfer Learning	0.667	0.656	0.705
Phase 2	Autoencoder (Unsupervised)	Unsupervised	0.550	0.130	0.480
Phase 3	BiLSTM (rows as sequences)	Recurrent (LSTM)	~Run-time	~Run-time	~Run-time
Phase 3	ViT-Base/16 (fine-tuned)	Transformer	~Run-time	~Run-time	~Run-time

Table 11: Full Phase 2 vs Phase 3 Comparison (Phase 3 metrics computed at runtime from test set)

Comparison Visualizations Generated:

- **Final Comparison Bar Chart** (phase3_final_comparison.png): Grouped bar chart showing Accuracy, F1, and AUC for all 5 models, separated by Phase 2 (blue) and Phase 3 (red) with a vertical dashed boundary line.
- **Combined ROC Curve** (phase3_roc_comparison.png): Full ROC curves for BiLSTM and ViT, with reference scatter points for Phase 2 CNN and VGG16 baselines.
- **Phase 3 Summary Print:** A detailed text summary is printed including delta values (Δ) comparing each Phase 3 model to the Phase 2 CNN baseline.

7. Model Architecture Diagrams & Code Analysis

7.1 EfficientNetB0 + BiRNN/BiGRU Pipeline

The following illustrates the complete data flow for the hybrid CNN-sequence models:

Stage	Output Tensor	Operation Details
Input X-ray	(224×224×3)	Grayscale X-ray converted to 3-channel RGB
EfficientNetB0	(7×7×1280)	ImageNet-pretrained feature extraction (336 layers)
Conv2D(256, 1×1)	(7×7×256)	Channel dimension reduction; BatchNorm applied
Reshape	(49×256)	Spatial feature map → linear patch sequence
Bidirectional RNN/GRU	(49×256)	Sequence modelling (forward + backward pass)
GlobalAveragePool1D	(256)	Temporal aggregation across all 49 patches
Dense(128) + BN + Drop(0.45)	(128)	Primary classification head layer
Dense(64) + Drop(0.30)	(64)	Secondary classification head layer
Dense(1, sigmoid)	(1)	Disease probability output ∈ [0, 1]

Table 12: EfficientNetB0 + BiRNN/GRU Data Flow

7.2 ViT Processing Pipeline

The Vision Transformer processes images in a fundamentally different way from CNNs:

Stage	Shape	Description
RGB Input	(224×224×3)	ImageNet-normalized RGB image
Patch Embedding	(197×768)	196 patches (16×16) + CLS token; linear projection to 768-d
Positional Encoding	(197×768)	Learned 1D positional embeddings added
Transformer Blocks 0–7 (frozen)	(197×768)	8 frozen blocks; general visual features preserved
Transformer Blocks 8–11 (trainable)	(197×768)	4 trainable blocks; domain adaptation
Layer Norm + CLS token	(768)	Extract CLS token representation after norm
Custom Head: LayerNorm(768)	(768)	Stabilize input to classification head
Linear(768→256) + GELU + Drop(0.4)	(256)	Dimensionality reduction
Linear(256→64) + GELU + Drop(0.2)	(64)	Compressed feature space
Linear(64→1)	(1)	Binary logit → sigmoid → disease probability

Table 13: ViT-Base/16 Processing Pipeline

7.3 Key Code Segments Analysis

Notebook 1 — Model Builder Function:

The build_cnn_sequence_model(sequence_layer) function accepts either 'rnn' or 'gru' as a parameter, constructing the appropriate bidirectional recurrent layer dynamically. This single-function design enables direct A/B comparison under identical conditions. The train_two_stage() function encapsulates the complete two-stage training protocol for either model variant.

Notebook 2 — ViT Hook-Based Attention Extraction:

The attention visualization uses PyTorch's **forward hook** mechanism (register_forward_hook) to intercept the output of attn_drop layers (attention dropout modules) within the last transformer block. This non-invasive approach does not require modifying the ViT source code. A critical implementation detail: fused_attn=False must be set on all attention modules to prevent fused attention kernels from bypassing the dropout hook.

Notebook 2 — Custom PyTorch Dataset:

The ChestXRayDataset class inherits from torch.utils.data.Dataset and implements __len__() and __getitem__(). Images are opened using PIL, converted to RGB, and transformed via torchvision transforms. This is wrapped in DataLoader with num_workers=2 for parallel data loading.

8. Discussion & Analysis

8.1 Architectural Design Decisions

■ **Why apply sequence models to 2D images?** The CNN+RNN/GRU approach exploits the spatial structure of convolutional feature maps: each location in the 7×7 feature map captures a different receptive field of the original X-ray. Treating these 49 spatial locations as a sequence allows the recurrent layer to model long-range spatial dependencies that CNNs with limited kernel sizes may not capture — effectively giving the model a form of spatial attention.

■ **GRU vs SimpleRNN:** GRU's gating mechanism (update gate + reset gate) provides a meaningful advantage over SimpleRNN for longer sequences by mitigating vanishing gradients. For 49-step sequences the difference may be modest, but GRU is generally preferred as a safe default for medical imaging applications where reliability matters.

■ **LSTM image row approach (Notebook 2):** Treating image rows as time steps is a creative application of sequence models to static images. The anatomical interpretation (top-to-bottom scanning of chest anatomy) provides clinical intuition. However, this approach loses 2D spatial context — a row's pixels are fed as a flat feature vector without positional encoding within the row.

■ **ViT for chest X-rays:** The global self-attention mechanism of ViT is well-suited for capturing long-range dependencies across distant anatomical regions (e.g., bilateral pulmonary findings). The attention visualization provides direct clinical interpretability — a key requirement for medical AI.

8.2 Training Strategy Analysis

■ **Two-stage fine-tuning (Notebook 1):** This approach follows established best practices for transfer learning on medical images. Stage 1 establishes a stable gradient landscape before Stage 2 partially relaxes the backbone, preventing the large gradient updates from the untrained head from corrupting pretrained weights.

■ **BatchNorm freezing during fine-tuning:** A critical technique for small medical datasets. Unfreezing BatchNorm layers on a small medical dataset can cause the running statistics to shift catastrophically, degrading performance. The notebook correctly freezes all BatchNorm layers throughout fine-tuning.

■ **Class weight compensation:** All models use sklearn's balanced class weights to penalize misclassification of the minority class more heavily. This is essential because the NIH sample has a class imbalance (~60% Normal), and without weighting, models may learn to predominantly predict "Normal".

■ **EarlyStopping on AUC:** Using AUC rather than accuracy or loss as the early stopping metric is appropriate for imbalanced classification tasks, as AUC is invariant to class prior distribution shifts.

8.3 Evaluation Framework Analysis

The evaluation framework across both notebooks is comprehensive and consistent. The choice of AUC-ROC as the primary metric is well-motivated for imbalanced binary medical classification: it measures the model's ability to rank positive cases above negative cases across all classification thresholds, making it independent of the operating threshold chosen at deployment.

F1-Score provides a complementary perspective by evaluating the balance between Precision (avoiding false alarms in disease detection) and Recall (avoiding missed disease cases — the more clinically critical error). In a screening context, high Recall (sensitivity) is typically prioritized over Precision.

9. Limitations & Future Work

9.1 Current Limitations

• **Binary Classification Simplification:** The task collapses 14 distinct pathology classes into a single "Disease" label. This prevents the system from distinguishing between, for example, pneumonia and cardiomegaly — a critical distinction for clinical decision-making.

• **Sample Size:** Notebook 2 limits training to 3,000 images (out of 5,606 available). Small datasets can lead to high variance in evaluation metrics and may underfit complex architectures like ViT.

• **No Test-Time Augmentation (TTA):** None of the models apply TTA during inference, which could improve prediction stability and AUC scores.

• **LSTM Spatial Information Loss:** Treating image rows as time steps discards the 2D spatial relationships between pixels within each row and between different columns across rows. A more principled approach would use 2D-LSTMs or ConvLSTM layers.

• **No Multi-Label Evaluation:** The NIH dataset supports multi-label annotation (a patient may have both Atelectasis and Effusion), but the binary framing ignores this richness.

• **Attention Approximation:** The CLS-token attention averaged across heads is an approximation of true model attention. More rigorous methods include Attention Rollout or GradCAM applied to specific attention layers.

9.2 Future Work

■ **Multi-label classification:** Extend to 14-class multi-label prediction using sigmoid activation with per-class binary cross-entropy.

■ **Full dataset training:** Use the complete NIH Chest X-Ray dataset (112,120 images) for training.

■ **DeiT or Swin Transformer:** Explore data-efficient ViT variants (DeiT) designed for smaller datasets, or hierarchical vision transformers (Swin) with local attention windows.

■ **Hybrid ViT-CNN architectures:** Combine CNN feature extraction with transformer attention (e.g., using EfficientNet as a patch feature extractor feeding ViT blocks).

■ **Clinical validation:** Compare model predictions against radiologist annotations on a held-out prospective cohort.

■ **Explainability:** Apply Grad-CAM, SHAP, or Attention Rollout for more faithful attribution maps that can be validated against radiological ground truth.

■ **Deployment:** Package the best-performing model as a clinical decision support tool with a confidence-calibrated output and uncertainty estimation.

10. Conclusion

Phase 3 of the Medical Image Diagnosis System project successfully implements and evaluates four advanced deep learning architectures for automated chest X-ray pathology detection on the NIH Chest X-Ray dataset. The work is distributed across two Jupyter notebooks that collectively cover:

- ✔ A hybrid **EfficientNetB0 + Bidirectional RNN/GRU** architecture with two-stage fine-tuning, demonstrating how spatial feature maps from pretrained CNNs can be effectively repurposed as sequences for recurrent processing.
- ✔ A pure **Bidirectional LSTM** model treating X-ray image rows as temporal sequences — a creative and computationally efficient approach to applying sequence models to medical images.
- ✔ A fine-tuned **Vision Transformer (ViT-Base/16)** using PyTorch and timm, representing the state-of-the-art in image recognition and offering interpretable attention-based features.
- ✔ **Attention heatmap visualization** that reveals which anatomical regions the ViT attends to during classification — a critical step toward clinically trustworthy AI.
- ✔ A **comprehensive cross-phase evaluation** comparing all Phase 2 and Phase 3 models using standardized metrics (Accuracy, F1-Score, AUC-ROC) on identical test sets.

The phase demonstrates the practical application of modern deep learning paradigms — from classical recurrent sequence models to attention-based transformer architectures — to a high-stakes medical imaging task. The consistent evaluation framework and reproducible code design enable direct comparison across all architectural variants, providing a solid foundation for future extensions toward multi-label classification and clinical deployment.

Key Achievement: Phase 3 extends the project from convolutional networks (Phase 2) into the domains of sequence modeling and self-attention, covering the three major families of modern deep learning architectures for image classification: CNNs, RNNs/LSTMs, and Transformers.

11. References & Technical Stack

11.1 Dataset

• Wang, X. et al. (2017). "ChestX-ray8: Hospital-scale Chest X-ray Database and Benchmarks." CVPR 2017.

• NIH Clinical Center. "NIH Chest X-ray Dataset." Available at: https://nihcc.app.box.com/v/ChestXray-NIHCC

• Kaggle Dataset: nih-chest-xrays/sample (5,606 sample images with labels CSV)

11.2 Model References

• Tan, M. & Le, Q.V. (2019). "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks." ICML 2019.

• Dosovitskiy, A. et al. (2020). "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale." ICLR 2021.

• Hochreiter, S. & Schmidhuber, J. (1997). "Long Short-Term Memory." Neural Computation, 9(8), 1735-1780.

• Cho, K. et al. (2014). "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation." EMNLP 2014.

• Simonyan, K. & Zisserman, A. (2014). "Very Deep Convolutional Networks for Large-Scale Image Recognition." ICLR 2015.

11.3 Software & Framework Stack

Library / Tool	Version	Usage in Project
TensorFlow / Keras	≥2.10	EfficientNetB0, BiRNN, BiGRU, BiLSTM models; training pipeline
PyTorch	≥2.0	ViT model implementation, custom training loop, DataLoader
timm	Latest	Pretrained ViT-Base/16 model loading and fine-tuning
torchvision	≥0.15	Image transforms, normalization, augmentation for ViT
scikit-learn	≥1.0	Metrics (AUC, F1, accuracy), class weight computation, train/test split
NumPy	≥1.23	Numerical array operations throughout
Pandas	≥1.5	DataFrame management for labels, file paths, results
Matplotlib	≥3.5	Training curves, ROC curves, attention heatmaps
Seaborn	≥0.12	Confusion matrix heatmaps, comparison bar charts
Pillow (PIL)	≥9.0	Image loading and format conversion
OpenCV (cv2)	≥4.5	Attention map bicubic resizing for ViT heatmaps
kagglehub	Latest	Automated NIH dataset download in Notebook 1

Table 14: Complete Technical Stack

11.4 Output Files Generated

File	Source Notebook	Description
best_rnn_nih_xray_stage1.keras	Notebook 1	Best BiRNN weights (Stage 1, by val_auc)
best_gru_nih_xray_finetuned.keras	Notebook 1	Best BiGRU weights (fine-tuned, by val_auc)
phase3_nih_xray_rnn_finetuned.keras	Notebook 1	Final saved BiRNN model
phase3_nih_xray_rnn_gru_results.csv	Notebook 1	BiRNN vs BiGRU metrics comparison CSV
lstm_training_curves.png	Notebook 2	BiLSTM loss/accuracy/AUC/precision-recall training curves
lstm_evaluation.png	Notebook 2	BiLSTM confusion matrix and ROC curve
vit_training_curves.png	Notebook 2	ViT loss/accuracy/AUC training curves
vit_evaluation.png	Notebook 2	ViT confusion matrix and ROC curve
vit_attention_heatmaps.png	Notebook 2	Normal vs Diseased attention heatmap overlays (3-row × 8-col)
vit_attention_heads.png	Notebook 2	Per-head attention visualization (12 heads, diseased sample)
phase3_final_comparison.png	Notebook 2	Phase 2 vs Phase 3 grouped bar chart comparison
phase3_roc_comparison.png	Notebook 2	Combined ROC curves for all Phase 2 & 3 models
phase3_results.csv	Notebook 2	Full cross-phase comparison metrics table
phase3_lstm_model.keras	Notebook 2	Final saved BiLSTM model
best_vit.pth	Notebook 2	Best ViT checkpoint (PyTorch state dict, by val_auc)
best_lstm.keras	Notebook 2	Best BiLSTM checkpoint (by val_auc)

Table 15: Complete Output Files — Phase 3

End of Report | Medical Image Diagnosis System — Phase 3 | Cairo University, Faculty of Computers and Artificial Intelligence | Spring 2026